OUTSMART STUDIO

GUIDA · V1.0

OUTSMART CMS — DOCUMENTAZIONE INTERNA

{Prova in locale}

Installa il motore. Carica il template. Provalo tutto prima di pubblicare.

LUGANO · SETTEMBRE 2026

OUTSMART SAGL

Tre cose accese. Poi si parte.

Il CMS è il **motore**: admin, database, login, immagini, moduli, pubblicazione. Il sito è un **template** che si installa a parte. Un'installazione, un sito, un database.

COSA	PERCHÉ
Node.js 22.12+	Il Mac usa di default una versione più vecchia. In ogni terminale nuovo: `. ~/.nvm/nvm.sh && nvm use 22`
Docker Desktop	Acceso. Ospita MySQL, sulla porta `3307`.
Cartella del progetto	`outsmart-cms`. Ogni comando di questa guida parte da lì.

Il database

```
npm run db:up      # avvia MySQL e aspetta che sia pronto
npm run db:down    # lo spegne, i dati restano nel volume
```

Due modi per provare

MODO A

Istanza di sviluppo

Quella già installata. Il modo più rapido per toccare admin, stile, immagini, moduli e template.

MODO B

Installazione da zero

Una copia separata, con il suo database. Serve a vedere la procedura guidata. Non tocca la A.

REGOLA

Le credenziali non si scrivono mai nei comandi né nei file del progetto. Stanno nel `.env` (ignorato da git) o nelle variabili d'ambiente.

Avvia. Accedi. Tocca tutto.

01 **Avvia l'app.** L'admin si apre su `http://localhost:3000/admin`.

```
npm run dev
```

02 **Accedi** con `info@out-smart.ch` e la password che hai scelto. Se non la ricordi, se ne genera una nuova. Compare *una sola volta*, alla riga «Password (mostrata una sola volta…)».

```
node scripts/create-user.mjs --email info@out-smart.ch \
  --name Outsmart --role admin --reset
```

03 **Vedi il sito pubblico**, in un secondo terminale. I moduli arrivano all'app sulla porta 3000, quindi l'invio funziona. Dopo ogni modifica ai contenuti usa *Pubblica* nell'admin: rigenera il sito.

```
npm run build     # genera il sito statico in dist/
npm run serve     # lo serve su http://localhost:4173
```

Cosa provare

DOVE	COSA
Pagine	Modifica testi e blocchi: l'anteprima cambia subito. Riordina, duplica, nascondi, aggiungi.
Stile	Colori, dimensioni dei titoli, effetto dei pulsanti, con anteprima dal vivo.
Immagini	Carica (vengono ricodificate in WebP) e scegli il punto focale.
Dati del sito	Contatti, menu, consenso alla mappa (integrato o iubenda).
Email	Server SMTP ed email di prova. Senza SMTP le richieste si salvano lo stesso.
Richieste	Compila un modulo dal sito pubblico e ritrovalo nell'elenco.
Template	Attiva **Starter**, con blocchi propri. I contenuti si sostituiscono con quelli di esempio, con copia di sicurezza in `data/backups/`.
Utenti	Nessuna pagina di gestione: si creano da riga di comando, ruolo `editor` o `admin`. L'editor vede solo pagine e blocchi condivisi.

> **ATTENZIONE A .NEXT**
>
> `npm run dev` e `npm start` usano la stessa cartella `.next`. Se passi dall'uno all'altro l'admin si rompe: fermalo e rifai `npm run build:admin`.

Una copia pulita. Zero rischi per la A.

Il modo automatico

Database e cartelle temporanei. Percorre tutti i passi: chiave, requisiti, database, primo admin, zip del template, template dall'admin, cambio di template, rimozione. Alla fine pulisce. Serve l'admin già compilato (`npm run build:admin`) e Docker acceso.

```
node scripts/e2e-install.mjs
```

A mano, per vedere il wizard

01 **Ferma la A** con Ctrl+C. Due copie in sviluppo nella stessa cartella non convivono.

02 **Crea un database vuoto.** La password di root la legge il container: non va scritta.

```
docker exec outsmart-cms-db sh -c 'mysql -uroot -p"$MYSQL_ROOT_PASSWORD" \
  -e "CREATE DATABASE outsmart_cms_prova \
      CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci; \
      GRANT ALL ON outsmart_cms_prova.* TO \"cms\"@\"%\";"'
```

03 **Prepara lo zip del template.** Finisce nella cartella del progetto, per esempio `mammano-1.0.0.zip` .

```
npm run template:pack -- templates/mammano
```

04 **Avvia una copia isolata.** Cartelle a parte e variabili del `.env` di sviluppo svuotate: altrimenti la procedura vede il database già installato e si crede installata.

```
P=/tmp/cms-prova
env DB_HOST= DB_PORT= DB_NAME= DB_USER= DB_PASSWORD= DB_ROOT_PASSWORD= \
    SETTINGS_SECRET= IP_HASH_SECRET= APP_ORIGIN= TRUST_PROXY= INSTALL_TOKEN= \
    CMS_TEMPLATE= CMS_TEMPLATE_DIR= SMTP_HOST= MAIL_FROM= \
    DATA_DIR=$P/data TEMPLATES_DIR=$P/templates ENV_FILE=$P/.env \
    SITE_OUT_DIR=$P/dist MEDIA_DIR=$P/media \
    npx next dev -p 3100
```

Apri /install e segui i passi.

Vai su `http://localhost:3100/install`. Cinque passi e il sito è pubblicato.

PASSO	COSA FAI
Chiave	La trovi nel terminale, alla riga «Chiave di installazione…», e in `/tmp/cms-prova/data/install-token`.
Requisiti	Devono essere tutti a posto: Node, immagini, permessi, cartelle scrivibili.
Database	Host `127.0.0.1`, porta `3307`, nome `outsmart_cms_prova`, utente `cms` e la `DB_PASSWORD` del tuo `.env`.
Primo admin	Nome, email, password di almeno 12 caratteri.
Template	Carica lo zip del passo 3 e indica l'indirizzo pubblico del sito. Si installano i contenuti di esempio e il sito viene pubblicato subito.

Finito, `/install` si chiude per sempre e la chiave non serve più. Accedi da `http://localhost:3100/admin`.

> **PASSWORD DEL DATABASE**
>
> Non può contenere `$ ' " \ ` #`: il file `.env` non li conserva in modo affidabile.

Pulizia

```
docker exec outsmart-cms-db sh -c 'mysql -uroot -p"$MYSQL_ROOT_PASSWORD" \
  -e "DROP DATABASE outsmart_cms_prova"'
rm -rf /tmp/cms-prova mammano-1.0.0.zip
npm run build:admin   # la copia in sviluppo ha sovrascritto .next
```

Il valore si dimostra, non si promette.

184 test automatici e 55 attacchi all'app in esecuzione. Ripetibili in qualsiasi momento.

COMANDO	COSA VERIFICA	SERVE
`npm test`	184 test: token, sessioni, contenuti, template, installazione, sicurezza	Docker (database di prova temporaneo)
`e2e-install.mjs`	Installazione da zero e gestione dei template	Admin compilato, Docker
`e2e-admin`, **`e2e-editor`**, **`e2e-media`**, **`e2e-forms`**	L'admin con un browser vero	App avviata con `npm start`, utente admin temporaneo, `E2E_URL`, `E2E_EMAIL`, `E2E_PASSWORD`
`e2e-consent.mjs`	Consenso alla mappa sul sito statico	Niente
`npm run probe:security`	55 attacchi all'app in esecuzione	Come le prove nel browser

> **SALVANO E PUBBLICANO DAVVERO**
>
> Le prove nel browser modificano i contenuti e poi li ripristinano. Usale solo sul database di sviluppo.

L'utente temporaneo

```
node scripts/create-user.mjs --email e2e@esempio.test \
  --name E2E --role admin --reset
# … prove …  poi lo elimini dal database:
docker exec outsmart-cms-db sh -c 'mysql -uroot -p"$MYSQL_ROOT_PASSWORD" outsmart_cms \
  -e "DELETE FROM users WHERE email=\"e2e@esempio.test\""'
```

Errori noti. Con la causa.

MESSAGGIO IN CONSOLE	COSA SIGNIFICA
«unsafe-eval» in sviluppo	Risolto. Con `npm run dev` la politica di sicurezza ammette `eval` solo in quella modalità: serve al refresh automatico di React. In produzione no.
«...from origin 'null' ... _devMiddlewareManifest.json»	Una pagina dell'admin era finita dentro l'iframe dell'anteprima, che ha origine `null` perché isolato. Ora l'anteprima riceve HTML dall'editor e, senza sessione, `/admin/preview` risponde 401. Se lo rivedi: ricarica (la sessione può essere scaduta) e segnala pagina e browser.
«Failed to load resource» per `favicon.ico`	Innocuo. L'admin non ha un'icona del sito.
L'admin si rompe passando da dev a start	Stessa cartella `.next`. Rifai `npm run build:admin`.

Cosa NON è stato provato

Coerente con come lavoriamo: meglio dirlo prima che scoprirlo dopo.

- L'installazione su un server vero (Infomaniak).
- L'invio email con un SMTP reale.
- iubenda con un account vero: finora solo uno script finto.

> **SICUREZZA DEI TEMPLATE**
>
> Un template è codice. Il suo isolamento ha limiti dichiarati (rete e segnali non sono limitati da Node; il JavaScript del template gira nel sito pubblico). Prima di installare template di terzi leggi «Template e sicurezza» in `docs/architettura.md`.

OUTSMART STUDIO

Tu pensi in grande.
Noi mettiamo le mani.

OUTSMART SAGL · LUGANO, SVIZZERA
OUT-SMART.CH · GABRIELE.MARELLI@OUT-SMART.CH